StanfordQuadruped 第 3 讲讲义：足端规划、Raibert 落脚点与逆运动学

这份讲义对应系列课程的第 3 讲，目标是把“机身速度命令如何变成足端目标，再如何变成关节角”这条链真正讲透。

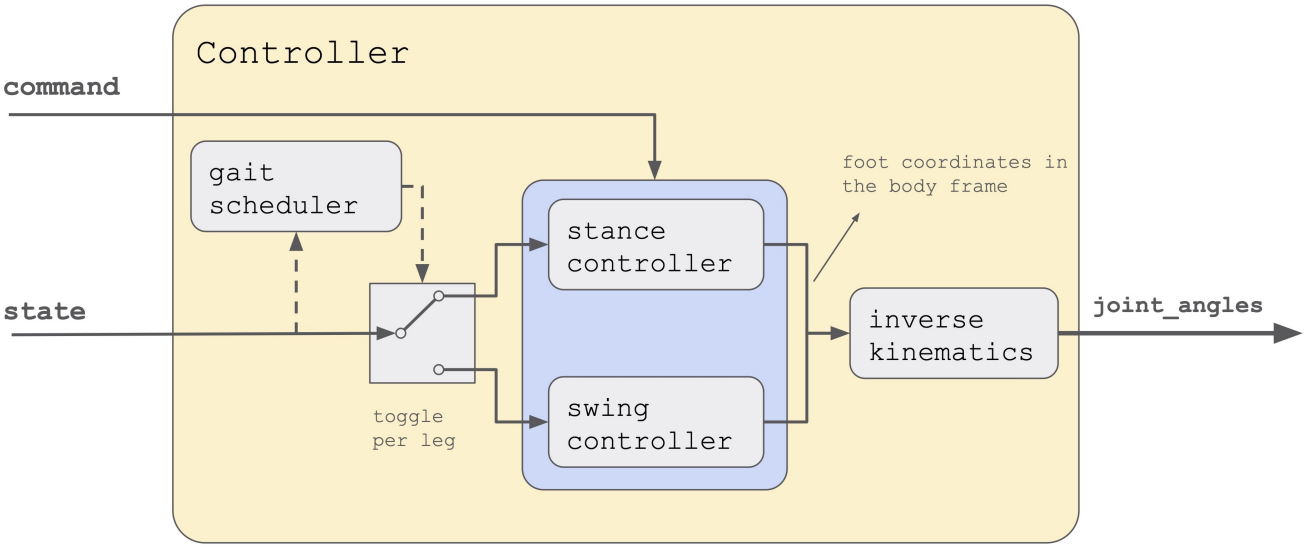
建议时长：40~50 分钟

主文件：

- `src/StanceController.py`
- `src/SwingLegController.py`
- `pupper/Kinematics.py`
- `src/Controller.py`

本讲目标

- 看懂 `StanceController` 和 `SwingController` 的几何意义
- 把“机身速度命令”翻译成“足端相对机身应该怎么动”
- 讲清楚 Raibert 落脚点启发式
- 讲清楚 `pupper/Kinematics.py` 怎样把足端位置变成关节角



控制器内部拆解图：gait、stance、swing、IK 四块核心组件

先讲清一个关键坐标直觉

这个项目里的足端规划，大部分都在“机身坐标系下”进行。

所以观众必须理解：

- 控制器不是先求世界坐标系里的脚轨迹
- 而是先维护“脚相对于机身”的位置
- 真正要让机器人前进时，脚在机身坐标里往后划，身体在世界里才会向前走

‹ StanceController › 的直觉

支撑相里，脚在地上不想乱飞，所以它做的是“ 相对机身向后划地 ”。

代码里的核心直觉可以压缩成：

```
期望机身前进
 -> 足端在机身坐标中向后移动

期望机身转向
 -> 足端绕 z 轴做反向旋转

期望高度回到目标值
 -> 足端 z 方向缓慢收敛
```

如果要讲公式，最值得讲的是这三件事：

- ‹ v_xy = -command.horizontal_velocity ›
- ‹ delta_R = yaw_rate * dt › 对应的反向旋转
- ‹ z_time_constant › 用来把足端高度拉回目标高度

‹ SwingController › 的直觉

摆动相里，脚要完成三件事：

- 抬起来
- 向新的落脚点移动
- 按时落下

当前实现非常工程化：

- 抬脚高度走一个三角波
- ‹ x / y › 朝落脚点匀速逼近
- ‹ z › 直接由 ‹ swing_height + command.height › 给出

这意味着它不是最平滑的高阶轨迹，但非常清楚、容易教、容易改。

Raibert 落脚点一定要讲成“ 工程启发式 ”

‹ raibert_touchdown_location() › 的意义是：

- 机器人前进越快，落脚点越应该往前放
- 转向越快，落脚点越应该做额外偏航旋转补偿

代码中最值得讲的两个系数：

- ‹ alpha ›
- 决定水平速度对落脚点前后偏移的影响
- ‹ beta ›
- 决定偏航速度对落脚点旋转补偿的影响

可以引导观众记住一句话：

> Raibert 不是“ 精确最优控制 ”，而是一个在足式机器人里非常经典、非常实用的落脚点启发式。

`‹ pupper/Kinematics.py ›` 怎么讲最顺

建议按“ 先减去腿原点偏移，再解单腿 IK ”的顺序讲。

四腿 IK 的结构其实很整齐：

```
四足足端矩阵 (3x4)
 -> 每条腿减去自身安装原点
 -> 单腿解析逆解
 -> 输出 3x4 关节角矩阵
```

单腿逆解重点不是推满全部三角关系，而是讲清几个几何量：

- 足端在 `‹ y-z ›` 平面的投影距离
- 外展偏移 `‹ ABDUCTION_OFFSET ›`
- 髋到足端的空间距离
- 由余弦定理得到的髋角和膝角

如果要讲成视频，建议把难点压成一句话：

> 这套 IK
> 不是数值优化，而是解析几何；它依赖已知的机身尺寸、腿长和外展偏移，直接算三个关节角。

`‹ Controller ›` 里的姿态补偿不要漏掉

在 `‹ TROT ›` 分支里，控制器会根据 IMU 的 `‹ roll / pitch ›` 做倾斜补偿：

- 先从四元数提取欧拉角
- 再对 `‹ roll / pitch ›` 做裁剪
- 乘以固定系数后，反向作用到足端目标

这里可以顺带告诉观众：

- 这是“ 轻量姿态补偿 ”，不是完整姿态控制器
- 作用是减弱机身倾斜对足端目标的破坏
- 它非常适合作为“ 闭环味道 ”的入门示例

本讲建议演示

先做固定机身，便于观众只看足端与 IK：

```
python sim/run_fixed_base.py --mode rest --pitch 0.15 --roll 0.10
python sim/run_fixed_base.py --mode trot --x-vel 0.15 --yaw-rate 0.6
```

再演示浮动机身里 `‹ z_clearance ›` 的影响：

```
python sim/run_floating_base.py --duration 8 --mode trot \
  --z-clearance 0.05 --no-plots
```

本讲作业

- 让观众解释“ 为什么机身前进时，支撑足要在机身坐标里往后走 ”
- 让观众解释 `‹ alpha ›` 变大后落脚点会发生什么变化

- 让观众自己画一条摆动相三角波高度轨迹